

```
import random

# Processing time of 5 jobs
jobs = [4, 2, 7, 3, 5]

# Number of particles
N = 10

# Initial population
particles = []
for _ in range(N):
    p = list(range(len(jobs)))
    random.shuffle(p)
    particles.append(p)

# Fitness: total completion time
def fitness(p):
    time = 0
    total = 0

    for i in p:
        time += jobs[i]
        total += time

    return total

# PSO iterations
best = min(particles, key=fitness)
best_value = fitness(best)

for _ in range(50):
```

for p in particles:

Move particle by swapping two jobs

i, j = random.sample(range(len(jobs)), 2)

p[i], p[j] = p[j], p[i]

Update global best

if fitness(p) < best_value:

best = p.copy()

best_value = fitness(p)

print("Best Job Schedule:", best)

print("Minimum Completion Time:", best_value)

```
particles.append(p)

# Fitness: total completion time
def fitness(p):
    time = 0
    total = 0

    for i in p:
        time += jobs[i]
        total += time

    return total

# PSO iterations
best = min(particles, key=fitness)
best_value = fitness(best)

for _ in range(50):

    for p in particles:

        # Move particle by swapping two jobs
        i, j = random.sample(range(len(jobs)), 2)
        p[i], p[j] = p[j], p[i]

        # Update global best
        if fitness(p) < best_value:
            best = p.copy()
            best_value = fitness(p)

    print("Best Job Schedule:", best)
    print("Minimum Completion Time:", best_value)
```

```
... Best Job Schedule: [1, 3, 0, 4, 2]
    Minimum Completion Time: 51
```